Problem1:

```
#include <iostream>
using namespace std;
int l;
class node
{
public:
   int data;
   node* leftChild ;
   node* rightChild;
   node(char c){
      this->data = c;
      leftChild = rightChild = NULL;
   };
   ~node(){

   };
};

class tree
{public:
   node* root ;

   tree(){
      this->root = NULL;
   }
```

```
bool treeIsEmpty(){
   if (root == NULL)
   {
     return true;
   }
   else{
      return false;
   }

}

void insertIntree(char c){
   if (root == NULL)
   {
      node* n = new node(c);
      n->leftChild=NULL;
      n->rightChild = NULL;

   }

   else{
      node* temp = root;
      while (true)
      {

         if (temp->data < c  && temp->rightChild ==NULL)
```

```
  {
     node* n = new node(c);
     temp->rightChild= n;
     break;

  }
   else if(temp->data < c  && temp->rightChild !=NULL){          //right child
     temp  =temp->rightChild;

  }


  else if (temp->data > c && temp->leftChild == NULL )
  {
     node* n = new node(c);
     temp->leftChild = n;
     break;
  }
  else if (temp->data > c && temp->leftChild != NULL )       //left child
  {
     temp = temp->leftChild;
  }


}
```

```
    }



}



 node* noOFnodes(node* r, int &a){
    if (root == NULL)
    {
       return NULL;
    }

    a++;
    noOFnodes(r->leftChild , a); // left child
    noOFnodes(r->rightChild , a); // right child
}

int NodeCounter(node* r){
    int a =  0;
    node* x = noOFnodes(r, a);
    return a;
}

 node* Leafs(node* r, int &a){     //leafNOdes- counter
    if (root == NULL)
    {
```

```
    return NULL;
  }

  if (root->leftChild == NULL && root->rightChild == NULL)
  {
    a++;
  }

  Leafs(r->leftChild , a); // left child
  Leafs(r->rightChild , a); // right child
}

int noofLeaf(node* r){
  int a =  0;
 node* x = Leafs(r, a);
  return a;
}

 node* srch(node* root, bool &isFound , int n){
  if (root == NULL)
  {
    return NULL;
  }

  if (root->data == n)
  {
```

```
      isFound = true;
    }

    srch(root->leftChild ,isFound , n); // left child
    srch(root->rightChild , isFound , n); // right child
}

bool IsFoundinTree(char c){
    bool isF = false;
    int cc = int(c);
    node* x = srch(this->root , isF ,cc );
    return isF;

}

node* PreOrder(node* root){
    if (root == NULL)
    {
      return NULL;
    }

    cout << root->data << " ";
    PreOrder(root->leftChild); // left child
    PreOrder(root->rightChild); // right child
}
node* PostOrder(node* root){
```

```
    if (root == NULL)
    {
        return NULL;
    }


    PostOrder(root->leftChild); // left child
    PostOrder(root->rightChild); // right child
    cout << root->data << " ";
}
node* Inorder(node* root){
    if (root == NULL)
    {
        return NULL;
    }


    Inorder(root->leftChild); // left child
    cout << root->data << " ";
    Inorder(root->rightChild); // right child

}

    node* del(node* root){
    if (root == NULL)
    {
```

```
        return NULL;
      }


      del(root->leftChild); // left child
      del(root->rightChild); // right child

      node* remp = root;
      delete remp;
   };

   ~tree(){
      node* root = del(root);
   }


};

int main(){
tree t;
t.insertIntree('f');
t.insertIntree('d');
t.insertIntree('p');
node* x = t.PreOrder(t.root);
node* x = t.PostOrder(t.root);
node* x = t.Inorder(t.root);
```

```
cout <<"is tree Empty " << t.treeIsEmpty() << endl;


if (t.IsFoundinTree('d'))
{
  cout << "Is Found in tree " << endl;
}
else{
   cout << "Is not Found in tree " << endl;
}


cout<< "The no of leaf Nodes "<<t.noofLeaf(t.root);
cout<< "The no of Nodes "<<t.NodeCounter(t.root);


   return 0;
}
```

Problem 2:

```
#include <iostream>
using namespace std;
int l;
class node
{
public:
   int data;
   node* leftChild ;
   node* rightChild;
```

```
  node(char c){
    this->data = c;
    leftChild = rightChild = NULL;
  };
  ~node(){

  };
};

class tree
{public:
  node* root ;

  tree(){
    this->root = NULL;
  }

  bool treeIsEmpty(){
    if (root == NULL)
    {
      return true;
    }
    else{
       return false;
    }
```

```
}

void insertIntree(char c){
   if (root == NULL)
   {
      node* n = new node(c);
      n->leftChild=NULL;
      n->rightChild = NULL;

   }

   else{
      node* temp = root;
      while (true)
      {

         if (temp->data < c  && temp->rightChild ==NULL)
        {
            node* n = new node(c);
            temp->rightChild= n;
            break;

        }
         else if(temp->data < c  && temp->rightChild !=NULL){          //right child
            temp  =temp->rightChild;
```

```
        }



        else if (temp->data > c && temp->leftChild == NULL )
        {
           node* n = new node(c);
           temp->leftChild = n;
           break;
        }
        else if (temp->data > c && temp->leftChild != NULL )       //left child
        {
           temp = temp->leftChild;
        }



      }


   }



}



 node* noOFnodes(node* r, int &a){
   if (root == NULL)
   {
```

```
    return NULL;
  }

  a++;
  noOFnodes(r->leftChild , a); // left child
  noOFnodes(r->rightChild , a); // right child
}

int NodeCounter(node* r){
  int a =  0;
  node* x = noOFnodes(r, a);
  return a;
}

 node* Leafs(node* r, int &a){     //leafNOdes- counter
  if (root == NULL)
  {
    return NULL;
  }

  if (root->leftChild == NULL && root->rightChild == NULL)
  {
    a++;
  }

  Leafs(r->leftChild , a); // left child
```

```
    Leafs(r->rightChild , a); // right child
}

int noofLeaf(node* r){
    int a =  0;
   node* x = Leafs(r, a);
    return a;
}

 node* srch(node* root, bool &isFound , int n){
    if (root == NULL)
    {
        return NULL;
    }

    if (root->data == n)
    {
        isFound = true;
    }

    srch(root->leftChild ,isFound , n); // left child
    srch(root->rightChild , isFound , n); // right child
}

bool IsFoundinTree(char c){
    bool isF = false;
```

```
    int cc = int(c);
    node* x = srch(this->root , isF ,cc );
    return isF;

}

node* PreOrder(node* root){
    if (root == NULL)
    {
        return NULL;
    }

    cout << root->data << " ";
    PreOrder(root->leftChild); // left child
    PreOrder(root->rightChild); // right child
}
node* PostOrder(node* root){
    if (root == NULL)
    {
        return NULL;
    }


    PostOrder(root->leftChild); // left child
    PostOrder(root->rightChild); // right child
    cout << root->data << " ";
```

```
}
node* Inorder(node* root){
   if (root == NULL)
   {
      return NULL;
   }


   Inorder(root->leftChild); // left child
   cout << root->data << " ";
   Inorder(root->rightChild); // right child

}

   node* del(node* root){
   if (root == NULL)
   {
      return NULL;
   }


   del(root->leftChild); // left child
   del(root->rightChild); // right child

   node* remp = root;
   delete remp;
```

```
  };
  node* mirror(node* root){
    if(root->leftChild == NULL && root->rightChild ==NULL ){
      return NULL;
    }
    mirror(root->leftChild);
    mirror(root->rightChild);

    swap(root->leftChild, root->rightChild);
  }
  ~tree(){
    node* root = del(root);
  }


};

int main(){
tree t;
t.insertIntree('f');
t.insertIntree('d');
t.insertIntree('p');
node* x = t.PreOrder(t.root);
node* x = t.PostOrder(t.root);
node* x = t.Inorder(t.root);
```

```
cout <<"is tree Empty " << t.treeIsEmpty() << endl;


if (t.IsFoundinTree('d'))
{
  cout << "Is Found in tree " << endl;
}
else{
   cout << "Is not Found in tree " << endl;
}


cout<< "The no of leaf Nodes "<<t.noofLeaf(t.root);
cout<< "The no of Nodes "<<t.NodeCounter(t.root);


t.Inorder(t.root);


t.mirror(t.root);
t.Inorder(t.root);
   return 0;
}
```

Problem 4:

```
#include <iostream>
using namespace std;
int l;
class node
{
public:
```

```
  int data;
  int size;
  node* leftChild ;
  node* rightChild;
  node(char c){
    this->data = c;
    leftChild = rightChild = NULL;
  };
  ~node(){

  };
};

class tree
{public:
  node* root ;

  tree(){
    this->root = NULL;
  }

  bool treeIsEmpty(){
    if (root == NULL)
    {
      return true;
    }
```

```
    else{
        return false;
    }

}

void insertIntree(char c){
    if (root == NULL)
    {
        node* n = new node(c);
        n->leftChild=NULL;
        n->rightChild = NULL;

    }

    else{
        node* temp = root;
        while (true)
        {

            if (temp->data < c  && temp->rightChild ==NULL)
            {
                node* n = new node(c);
                temp->rightChild= n;
                break;
```

```
        }
         else if(temp->data < c  && temp->rightChild !=NULL){          //right child
            temp  =temp->rightChild;


        }



        else if (temp->data > c && temp->leftChild == NULL )
        {
           node* n = new node(c);
           temp->leftChild = n;
           break;
        }
        else if (temp->data > c && temp->leftChild != NULL )       //left child
        {
           temp = temp->leftChild;
        }



      }


   }



}
```

```
 node* noOFnodes(node* r, int &a){
   if (root == NULL)
   {
      return NULL;
   }

   a++;
   noOFnodes(r->leftChild , a); // left child
   noOFnodes(r->rightChild , a); // right child
}

int NodeCounter(node* r){
   int a =  0;
   node* x = noOFnodes(r, a);
   return a;
}

 node* Leafs(node* r, int &a){     //leafNOdes- counter
   if (root == NULL)
   {
      return NULL;
   }

   if (root->leftChild == NULL && root->rightChild == NULL)
```

```
    {
      a++;
    }

    Leafs(r->leftChild , a); // left child
    Leafs(r->rightChild , a); // right child
}

int noofLeaf(node* r){
    int a =  0;
   node* x = Leafs(r, a);
    return a;
}

 node* srch(node* root, bool &isFound , int n){
    if (root == NULL)
    {
      return NULL;
    }

    if (root->data == n)
    {
      isFound = true;
    }

    srch(root->leftChild ,isFound , n); // left child
```

```
    srch(root->rightChild , isFound , n); // right child
}

bool IsFoundinTree(char c){
    bool isF = false;
    int cc = int(c);
    node* x = srch(this->root , isF ,cc );
    return isF;

}

node* PreOrder(node* root){
    if (root == NULL)
    {
        return NULL;
    }

    cout << root->data << " ";
    PreOrder(root->leftChild); // left child
    PreOrder(root->rightChild); // right child
}
node* PostOrder(node* root){
    if (root == NULL)
    {
        return NULL;
    }
```

```
    PostOrder(root->leftChild); // left child
    PostOrder(root->rightChild); // right child
    cout << root->data << " ";
}
node* Inorder(node* root){
    if (root == NULL)
    {
        return NULL;
    }


    Inorder(root->leftChild); // left child
    cout << root->data << " ";
    Inorder(root->rightChild); // right child

}

    node* del(node* root){
    if (root == NULL)
    {
        return NULL;
    }
```

```
    del(root->leftChild); // left child
    del(root->rightChild); // right child

    node* remp = root;
    delete remp;
  };

  node* computeSize(node* size){
      if ()
      {

      }

  }
  ~tree(){
    node* root = del(root);
  }


};

int main(){
tree t;
t.insertIntree('f');
t.insertIntree('d');
t.insertIntree('p');
```

```
node* x = t.PreOrder(t.root);

node* x = t.PostOrder(t.root);

node* x = t.Inorder(t.root);


cout <<"is tree Empty " << t.treeIsEmpty() << endl;


if (t.IsFoundinTree('d'))

{

  cout << "Is Found in tree " << endl;

}

else{

   cout << "Is not Found in tree " << endl;

}


cout<< "The no of leaf Nodes "<<t.noofLeaf(t.root);

cout<< "The no of Nodes "<<t.NodeCounter(t.root);


   return 0;

}
```

Problem5:

```
#include <iostream>
```

```
using namespace std;

class node {
public:
   char data;
   node* leftChild;
   node* rightChild;
   node(char c)  {
      this->data = c;
      leftChild = rightChild = NULL;
   }
   ~node() {}
};

class tree {
public:
   node* root;

   tree()  {
      root = NULL;
   }

   bool treeIsEmpty() {
      return (root == NULL);
   }
```

```
void insertIntree(char c) {
   if (root == NULL) {
      root = new node(c);
   } else {
      insertIntreeHelper(root, c);
   }
}

void insertIntreeHelper(node* r, char c) {
   if (r->data < c) {
      if (r->rightChild == NULL) {
         r->rightChild = new node(c);
      } else {
         insertIntreeHelper(r->rightChild, c);
      }
   } else {
      if (r->leftChild == NULL) {
         r->leftChild = new node(c);
      } else {
         insertIntreeHelper(r->leftChild, c);
      }
   }
}

void rangeSearch(char low, char high) {
   node* searchRange = root;
```

```
    char found = -1;
    while (searchRange != NULL) {
        if (searchRange->data >= low && searchRange->data <= high) {
            cout << searchRange->data << " ";
            found = 1;
        }
        if (searchRange->data < low) {
            searchRange = searchRange->rightChild;
        } else if (searchRange->data > high) {
            searchRange = searchRange->leftChild;
        } else {
            return;
        }
    }
    if (found == 0) {
        cout << "No products in the range [" << low << ", " << high << "]" << endl;
    }
}

bool treeIsEmpty() {
    return root == NULL;
}

int NodeCounter(node* r) {
    if (root == NULL) {
        return 0;
```

```
    }
    return 1 + NodeCounter(r->leftChild) + NodeCounter(r->rightChild);
}

int noofLeaf(node* r) {
    if (root == NULL) {
        return 0;
    }
    if (r->leftChild == NULL && r->rightChild == NULL) {
        return 1;
    }
    return noofLeaf(r->leftChild) + noofLeaf(r->rightChild);
}

void PreOrder(node* root) {
    if (root == NULL) {
        return;
    }
    cout << root->data << " ";
    PreOrder(root->leftChild);
    PreOrder(root->rightChild);
}

void PostOrder(node* root) {
    if (root == NULL) {
        return;
```

```
    }
    PostOrder(root->leftChild);
    PostOrder(root->rightChild);
    cout << root->data << " ";
}

void Inorder(node* root) {
    if (root == NULL) {
        return;
    }
    Inorder(root->leftChild);
    cout << root->data << " ";
    Inorder(root->rightChild);
}

node* del(node* root) {
    if (root == NULL) {
        return NULL;
    }
    node* remp = root;
    root = del(root->leftChild);
    root = del(root->rightChild);
    delete remp;
    return root;
}
```

```
    ~tree() {
        del(root);
    }
};

int main() {
    tree t;
    t.insertIntree('a');
    t.insertIntree('b');
    t.insertIntree('c');
    t.insertIntree('d');
    t.insertIntree('e');

    cout << "PreOrder traversal of the tree: ";
    t.PreOrder(t.root);
    cout << endl;

    cout << "PostOrder traversal of the tree: ";
    t.PostOrder(t.root);
    cout << endl;

    cout << "Inorder traversal of the tree: ";
    t.Inorder(t.root);
    cout << endl;

    cout << "Is tree Empty? " << t.treeIsEmpty() << endl;
```

```
    char low = 'a', high = 'e';
    cout << "Range of prices: " << low << " to " << high << endl;
    t.rangeSearch(low, high);

    cout << "Number of Nodes in the tree: " << t.NodeCounter(t.root) << endl;
    cout << "Number of Leaf Nodes in the tree: " << t.noofLeaf(t.root) << endl;

    return 0;
}
```